Team Name : Mohit

Team Leader Name : Mohit

Problem Statement : Intelligent Candidate Discovery & Ranking — rank a 100,000-candidate pool for Redrob's Senior AI Engineer JD

Solution Overview

What is your proposed solution?

- A deterministic, explainable ranking engine: score = fit × trust × behavioral × logistics.
- Fit is mined from what candidates DID (career-history text) — never from the self-declared skills list.
- Pure Python standard library: 100K candidates ranked in ~56 s on CPU, ~275 MB RAM, no network, no GPU.

What differentiates it from traditional candidate matching?

- Traditional matchers count keyword overlap; ours scores evidence of shipped retrieval / ranking / evaluation work.
- Keyword stuffers (non-engineering titles with AI skill lists) are gated to 5% of their score.
- A trust layer mathematically excludes honeypots — profiles with impossible claims — without special-casing.
- Availability-aware: dormant, unresponsive, or logistically un-hirable candidates are down-weighted.

JD Understanding & Candidate Evaluation

Key requirements extracted from the JD

• Must-have: production embeddings/vector retrieval, ranking/recsys at scale, evaluation rigor (NDCG, MRR, offline↔online correlation, A/B), strong Python.

• Profile: 5–9 yrs (ideal 6–8), product-company background, hands-on; Pune/Noida or Tier-1 India; short notice period.

• Anti-patterns: consulting-only careers, research-only, CV/speech-only, title-chasers, keyword stuffers.

Signals that matter most — evaluating fit beyond keyword matching

• Career descriptions (what they shipped) carry the weight; the skills list earns only weak, assessment-corroborated credit.

• Claim consistency (trust), last-active recency + recruiter response rate (reachability), logistics feasibility.

• Word-boundary phrase matching, so 'storage' never matches 'RAG' — no naive keyword hits.

Ranking Methodology

Retrieve, score, rank

• One streaming pass over candidates.jsonl; every candidate scored independently — O(N), 100K in ~56 s.

Models, algorithms, heuristics

• Fit: 6 phrase-evidence groups (retrieval, ranking, evaluation, scale, LLM stack, applied ML) + title family + soft 6–8 yr experience peak + product/domain/education/GitHub credits − JD anti-pattern penalties.

• Trust: count of profile impossibilities → multiplier cliff 1.0 → 0.10 (honeypots cluster at 3+ violations).

• Behavioral ×0.55–1.12 (recency, response rate, open-to-work, interview completion); Logistics ×0.70–1.10 (city tier, notice, relocation).

Combining signals into the final ranking

• final = fit × trust × behavioral × logistics — multiplicative gates: a candidate must clear all four.

• Scores rounded, then sorted by (−score, candidate_id) for spec-exact monotonicity and tie-breaking.

Explainability & Data Validation

How ranking decisions are explained

• Every factor is stored per candidate; --explain prints the factor-by-factor breakdown for any rank.

• Each CSV row's reasoning is assembled from that candidate's extracted facts plus its single most significant honest concern.

Preventing hallucinations

• No LLM in the ranking path — reasoning is templated only from observed profile facts, so unsupported claims are impossible by construction.

• Sentence frames rotate on md5(candidate_id): varied wording, fully reproducible.

Handling inconsistent, low-quality, or suspicious profiles

• Impossibility checks: more months of a tech than it has existed (e.g. 88 mo of Pinecone), claimed years the career span cannot contain, jobs predating the employer's founding, 'expert' skills with ~0 months of use.

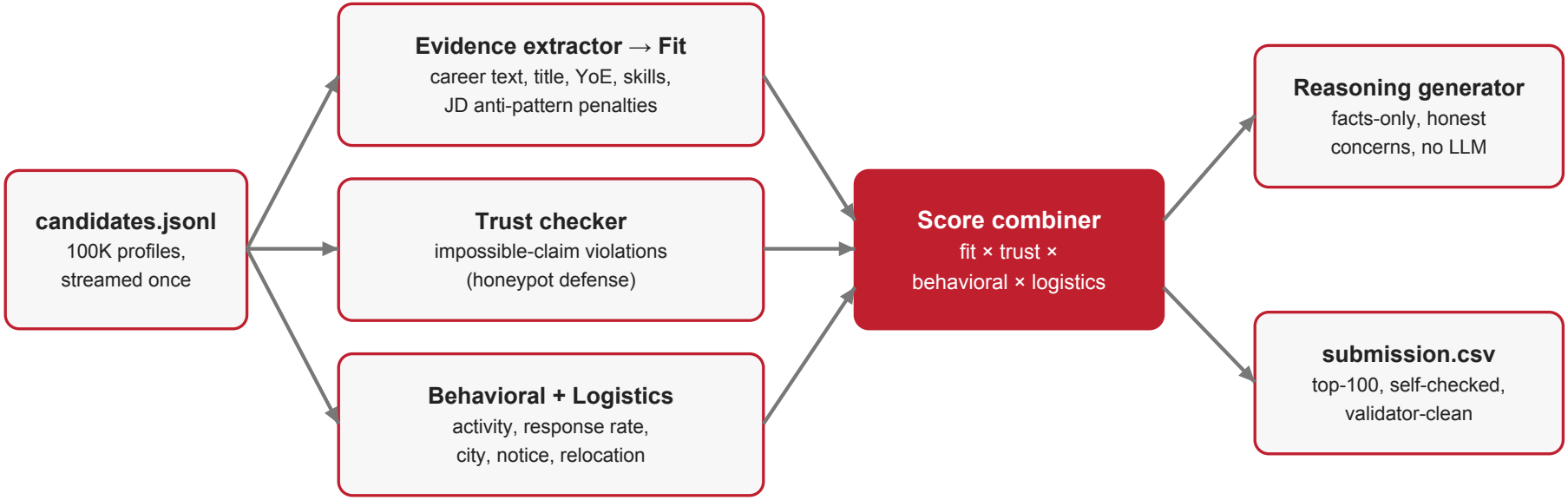
• Honest profiles carry 0–2 noisy violations; honeypots cluster at 3+ and fall off the trust cliff — 0 flagged profiles in our top-100.

End-to-End Workflow

From JD input to ranked candidate output

1. Stream candidates.jsonl (.gz supported) — one pass, constant memory.
2. Extract phrase evidence from career descriptions, summary, and headline.
3. Score structured fit: title family, experience band, corroborated skills, company background, JD anti-pattern penalties.
4. Count trust violations; compute behavioral and logistics multipliers.
5. Combine into the final score; round; sort by (−score, candidate_id).
6. Generate fact-based reasoning for each of the top 100.
7. Write the CSV → built-in self-check (honeypot histogram, gated titles, score range) → official validator passes.

System Architecture



Single deterministic pass, Python stdlib only — every stage CPU-cheap and auditable; multiplicative gates mean a candidate must be a real fit AND credible AND reachable AND hirable.

Results & Performance

Results demonstrating ranking quality

- Top-100: 92% inside the JD's 5–9 yr band, all ML/search/recsys titles, 96% India-based.
- 0 honeypot-flagged profiles in the top-100, verified by an independent recheck (disqualification threshold: >10%).
- Keyword-stuffer traps eliminated: the bundle's sample ranks HR Managers #1–2; ours ranks engineers who shipped search/recsys.
- 12 unit tests cover every trap; the output passes the official validate_submission.py.

Meeting the runtime and compute constraints

- ~56 s wall-clock and ~275 MB peak RAM for the full 100K pool (limits: 5 min, 16 GB).
- CPU-only, zero network, zero GPU — and deterministic: byte-identical CSV across reruns (md5-verified).

Technologies Used

Technologies and why they were selected

• Python 3 standard library only (json, csv, re, gzip, hashlib, datetime, unittest) — fits the 5-minute CPU budget, reproduces exactly in the Stage-3 Docker sandbox, zero dependency risk.

• Deliberately NO embedding model or LLM in the ranking path: the deciding signals are categorical (shipped ranking work? consistent claims? reachable?) — phrase-level evidence captures them explainably at a fraction of the cost.

• Streamlit — hosted sandbox UI only (sandbox/app.py), running the exact same ranking pipeline on uploaded samples.

• Git + GitHub for iteration history; Claude used for AI-assisted development (declared in submission metadata).

Submission Assets

Everything reviewers need

- GitHub repo: https://github.com/Mohit-45/redrob-ranker
- Submission CSV: submission.csv — top-100, validator-clean.
- Hosted sandbox: https://redrob-ranker-mohit.streamlit.app — runs the exact ranking pipeline in the browser.
- Demo sample: examples/sample_200.jsonl — upload it in the sandbox for a one-click demo.
- Metadata: submission_metadata.yaml — team, compute environment, AI declaration, methodology summary.
- This deck: Google Slides (link-viewable).
- Demo video: (add link here if you record one).

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU